Report - Task 1 - Anastasia Korshunova

The assignment was implemented as a simple standalone Java class (using Java 17) and the standard Java Cryptography API. The entire implementation is contained in a single class, Assignment1, without external dependencies or third-party libraries.

All input data, including the three candidate AES-128 keys, the expected SHA-256 hash, the encrypted message, and the initialization vector (IV), were hardcoded as constants in the source code.

The program performs four cryptographic operations sequentially:

Step 1. Identify the Correct Symmetric Key

Method: findCorrectKey()

Each candidate key is converted from hexadecimal format into a byte array. The SHA-256 hash of each key is calculated and compared with the expected hash provided in the assignment.

Result Correct symmetric key:

95ce2d1a16fde76dee842c90f126aab9

The second candidate key matches the expected SHA-256 hash.

Step 2. Decrypt the AES-128-CBC Message

Method: decryptAesCbc()

The identified symmetric key is used to decrypt the encrypted message using AES-128 in CBC mode with the provided initialization vector and PKCS5Padding.

The decrypted bytes are converted into a UTF-8 string.

Result Decrypted plaintext:

Consensus keeps the ledger honest

Step 3. Generate an Elliptic Curve Key Pair

Implementation: main() KeyPairGenerator

A new asymmetric key pair is generated using the Elliptic Curve algorithm and the secp256r1 (P-256) curve.

The generated public key is exported in PEM and hexadecimal X.509 DER formats. The private key is used for signing and is not printed.

Result Elliptic Curve parameters:

secp256r1 (P-256)

Generated public key (PEM):

-----BEGIN PUBLIC KEY-----

MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEXq7QIlCGjw+118Bv2xTtzgVnLVBw

a8WKLZy279EDLxPG+3AWyB22Onrzb98R2zG4FdTHglnASjSf5QpXOfzzbQ==

-----END PUBLIC KEY-----

Step 4. Generate and Verify the Digital Signature

Implementation: main() Signature

The original decrypted plaintext bytes are digitally signed using the generated EC private key and the SHA256withECDSA algorithm.

The signature is then verified using the corresponding public key.

Result Digital signature algorithm:

SHA256withECDSA

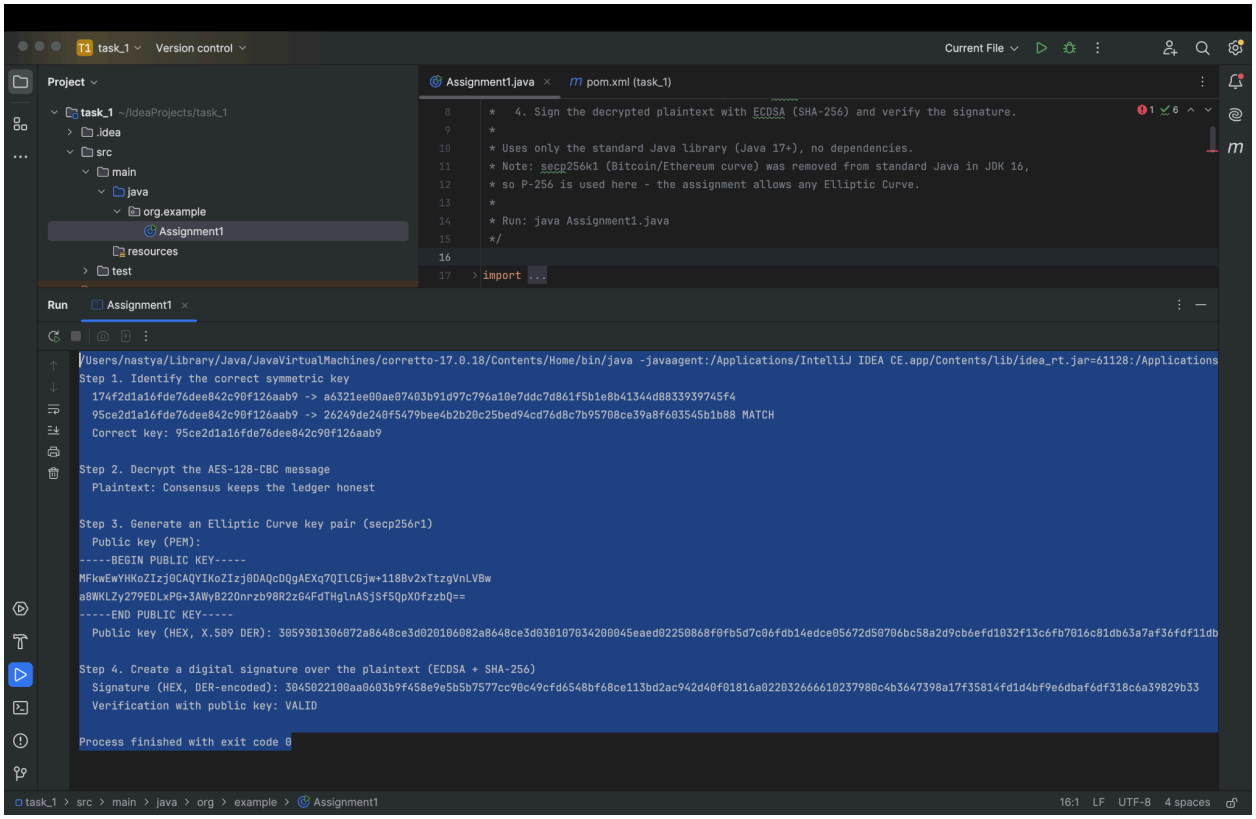
Generated digital signature (HEX, DER-encoded):

3045022100aa0603b9f458e9e5b5b7577cc90c49cfd6548bf68ce113bd2ac942d40f01816a022032666610237980c4b3647398a17f35814fd1d4bf9e6dbaf6df318c6a39829b33

Signature verification result:

VALID

The signature was successfully verified using the generated public key.



/Users/nastya/Library/Java/JavaVirtualMachines/corretto-17.0.18/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib/idea_rt.jar=61128:/Applications/IntelliJ IDEA CE.app/Contents/bin -Dfile.encoding=UTF-8 -classpath /Users/nastya/IdeaProjects/task_1/target/classes Assignment1

Step 1. Identify the correct symmetric key

174f2d1a16fde76dee842c90f126aab9 ->
a6321ee00ae07403b91d97c796a10e7ddc7d861f5b1e8b41344d8833939745f4

95ce2d1a16fde76dee842c90f126aab9 ->
26249de240f5479bee4b2b20c25bed94cd76d8c7b95708ce39a8f603545b1b88 MATCH

Correct key: 95ce2d1a16fde76dee842c90f126aab9

Step 2. Decrypt the AES-128-CBC message

Plaintext: Consensus keeps the ledger honest

Step 3. Generate an Elliptic Curve key pair (secp256r1)

Public key (PEM):

-----BEGIN PUBLIC KEY-----

MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEXq7QIlCGjw+118Bv2xTtzgVnLVBw

a8WKLZy279EDLxPG+3AWyB22Onrzb98R2zG4FdTHglnASjSf5QpXOfzzbQ==

-----END PUBLIC KEY-----

Public key (HEX, X.509 DER):
3059301306072a8648ce3d020106082a8648ce3d030107034200045eaed02250868f0fb5d7c06fdb14edce05672d5070
6bc58a2d9cb6efd1032f13c6fb7016c81db63a7af36fdf11db31b815d4c78259c04a349fe50a5739fcf36d

Step 4. Create a digital signature over the plaintext (ECDSA + SHA-256)

Signature (HEX, DER-encoded):
3045022100aa0603b9f458e9e5b5b7577cc90c49cfd6548bf68ce113bd2ac942d40f01816a022032666610237980c4b3
647398a17f35814fd1d4bf9e6dbaf6df318c6a39829b33

Verification with public key: VALID

Process finished with exit code 0